

CAPSTONE PROJECT OVERVIEW

Students are required to choose one of the following six research directions. The project must demonstrate not only technical implementation but also a deep understanding of Big Data architecture, optimization, and real-world problem-solving.

1. Real-time Data Integration & Stream Processing

- Description: Design and implement a pipeline to ingest high-velocity data from distributed sources, perform real-time cleaning/enrichment, and trigger immediate alerts or actions.
- Key Requirements:
 - Utilize Message Queues (e.g., Apache Kafka, RabbitMQ).
 - Implement windowing strategies (Sliding, Tumbling Windows).
 - Ensure data integrity using Exactly-once processing semantics.
 - Benchmark system latency under varying data loads.
- Application Themes: Real-time Fraud Detection or Cybersecurity Monitoring.
- Suggested Data Sources: [PaySim Mobile Money](#) or [CIC-IDS2018 Intrusion Detection](#).

2. Data Lakehouse Architecture & Governance

- Description: Deploy a modern Lakehouse architecture that bridges the gap between Data Lakes (low-cost storage) and Data Warehouses (ACID transactions, high performance).
- Key Requirements:
 - Implement a Medallion Architecture: Bronze (Raw), Silver (Cleaned), and Gold (Aggregated) layers.
 - Apply Schema Enforcement and Data Versioning (Time Travel).
 - Automate Data Quality (DQ) rules at each ingestion stage.
- Application Themes: Multi-channel Customer Data Platform (CDP).
- Suggested Data Sources: [Olist E-Commerce](#) or [Instacart Market Basket](#).

3. Knowledge Graph Integration for Heterogeneous Data

- Description: Integrate unstructured and structured data sources into a unified Knowledge Graph to discover hidden relationships between entities.
- Key Requirements:
 - Design a domain-specific Ontology/Schema.
 - Solve the Entity Resolution problem (deduplication of noisy data).

- Perform complex graph queries (e.g., shortest path, community detection).
- Application Themes: Medical Diagnosis Support or Global Supply Chain Analysis.
- Suggested Data Sources: [MIMIC-III Medical Database](#) or [DBpedia Datasets](#).

4. Privacy-Preserving Big Data Processing

- Description: Research and implement techniques to protect sensitive personal information while maintaining data utility for large-scale analytics.
- Key Requirements:
 - Implement Differential Privacy (ϵ -privacy) for statistical queries.
 - Apply anonymization methods (K-anonymity, L-diversity).
 - Evaluate the trade-off between privacy levels and analytical accuracy.
- Application Themes: Inter-bank Credit Scoring or Collaborative Healthcare Research.
- Suggested Data Sources: [Adult Census Income](#) or [IPUMS Community Survey](#).

5. Distributed Query Optimization on Columnar Storage

- Description: Analyze performance bottlenecks in Petabyte-scale queries and propose optimization strategies for distributed storage.
- Key Requirements:
 - Compare performance across formats (Parquet, ORC, Avro).
 - Optimize Partitioning and Bucketing to maximize data skipping.
 - Address Data Skew issues in large-scale distributed JOIN operations.
- Application Themes: Telecom Event Log Analysis or Large-scale GPS Trajectory Processing.
- Suggested Data Sources: [NYC Taxi Trip Records](#) or [Criteo 1TB Click Logs](#).

6. Hybrid Edge-Cloud Integration for IoT Ecosystems

- Description: Design a tiered processing architecture that optimizes the computational load between Edge devices and the Central Cloud.
- Key Requirements:
 - Implement lightweight protocols (MQTT or CoAP).
 - Perform Edge-side pre-processing (filtering/compression) to save bandwidth.
 - Manage Data Synchronization for intermittent network connectivity.
- Application Themes: Smart Factory Monitoring or Urban Traffic Management.
- Suggested Data Sources: [NASA Turbofan Degradation](#) or [Microsoft T-Drive GPS Sample](#).

General Grading Rubric Guidelines

Criteria	Weight	Requirements for Excellence
System Analysis & Design	20%	Clear problem statement; Robust, scalable architecture; Proper selection of Big Data ecosystem components.
Technical Implementation	30%	Stable pipeline/application; Proficiency in tools (Spark, Kafka, NoSQL); Clean, well-documented, and version-controlled code.
Optimization & Research Depth	20%	Evidence of optimization (e.g., skew handling, latency reduction); Comparative analysis of different configurations/methods.
Evaluation & Testing	15%	Rigorous test cases; Quantitative performance evaluation (Latency, Throughput, Accuracy) supported by charts.
Report & Presentation	15%	Academic writing style; Confident delivery; Ability to defend technical choices during Q&A.